Assignment 1

Report

Processor: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Physical cores: 4 (Total logical CPUs: 8)
Operating System: Ubuntu 24.04.3 LTS (Codename: noble)
GCC version: 13.3.0

Name: Pavlos Tomazos **ID:** 1115202200188

Exercise 1.1

The objective of this exercise is to implement naive polynomial multiplication and experiment with different techniques to parallelize the algorithm. The naive algorithm is presented below:

```
procedure NAIVE(a, b, n)
    res ← allocatePolynomial(2n)
    for i = 1 to n do
        for j = 1 to n do
            res[i + j] ← a[i] * b[j]
        end for
    end for
    return res
end procedure
```

A first attempt to parallelize the multiplication was to split the outer loop among $num_threads$, assigning each thread a portion of the work. To ensure correctness, a locking mechanism was necessary. However, as expected, the performance was not satisfactory due to the overhead of locking and unlocking, which effectively serialized the parallel computation.

```
procedure THREADWORK1(a, b, n)
    start ← ⌊((thread_id − 1) · n) / num_threads⌋ + 1
    end ← ⌊(thread_id · n) / num_threads⌋
    for i = start to end do
        for j = 1 to n do
            mutex ← lock
            res[i + j] += a[i] * b[j]
            mutex ← unlock
        end for
    end for
    return res
end procedure
```

Even with the following approach, for large n, the parallel algorithm performs worse than the sequential version. This is mainly because allocating a polynomial of size $2n$ for each thread dominates the total computation time, rather than the multiplication itself.

```
procedure ThreadWork2(a, b, n)
    temp ← allocatePolynomial(2n)
    start ← ⌊((thread_id − 1) · n) / num_threads⌋ + 1
    end ← ⌊(thread_id · n) / num_threads⌋
    for i = start to end do
        for j = 1 to n do
            temp[i + j] += a[i] * b[j]
        end for
    end for
    mutex ← lock
    for k = 1 to 2n do
        res[k] ← res[k] + temp[k]
    end for
    mutex ← unlock
    return res
end procedure
```

As visualized in the image below, each (i, j) pair that corresponds to the same sum k forms a diagonal in the array representation of the double loop.

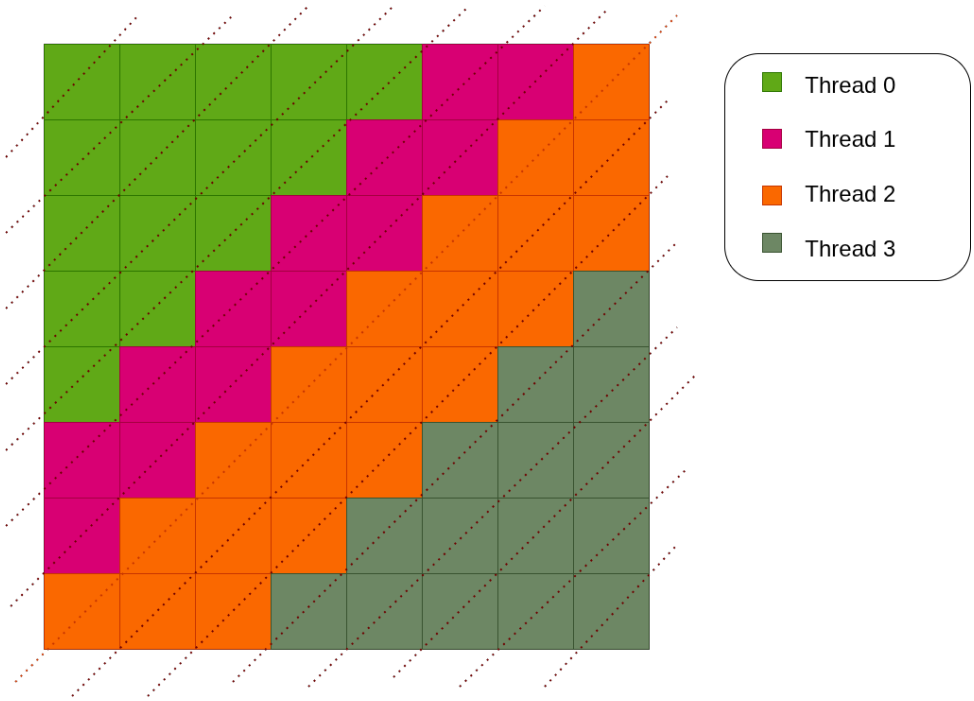


Figure 1: Sample visualization of work distribution among threads.

The idea is to assign each thread a set of diagonals. However, some sums correspond to more (i, j) pairs than others (i.e., diagonals of different lengths). Therefore, a preprocessing step is required to accumulate the total work and divide it evenly among threads.

This approach eliminates two major overheads in the parallel implementation: 1) unnecessary use of mutexes, and 2) allocation of temporary polynomials.

Algorithm 1 Distribute work among threads using a prefix array (descriptive version)

```
procedure DistributeWork(n, T)
    Compute a prefix array that stores the cumulative size of each diagonal
    This helps determine the amount of work contained in each diagonal
    Total work = sum of all diagonal sizes
    Compute the approximate amount of work per thread
    for t = 1 to T do
        Find the start diagonal for thread t
        Find the end diagonal for thread t
        Assign this range of diagonals to thread t
    end for
end procedure
```

Algorithm 2 Work done by each thread

```
procedure ThreadWork(a, b, result, start, end)
    for k = start to end do
        compute result[k] using a and b
    end for
end procedure
```

Analysis of Results

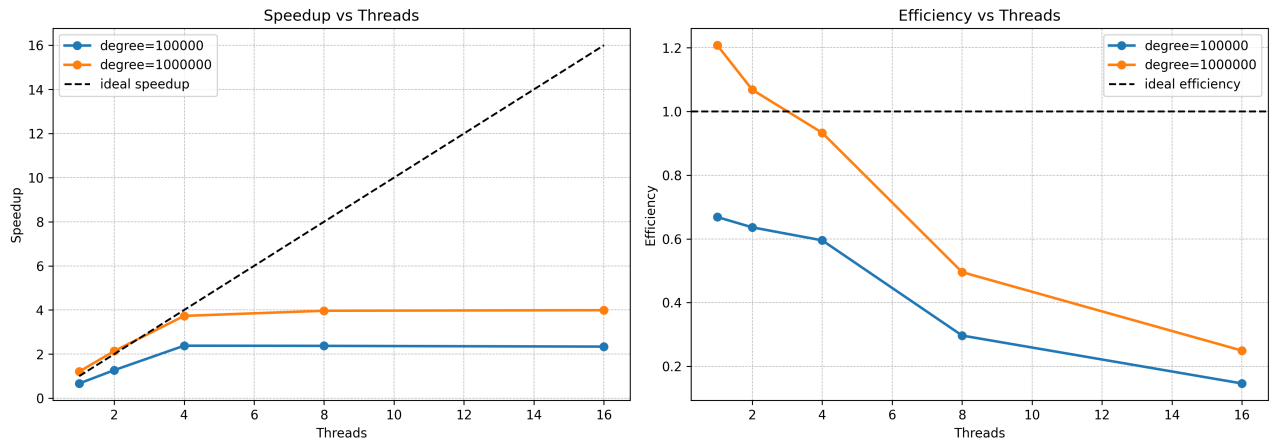


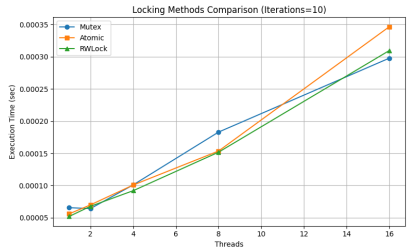
Figure 2: Speedup and Efficiency vs Number of Threads for different polynomial degrees.

The larger problem ($N = 10^6$, orange line) scales better than the smaller one ($N = 10^5$, blue line). With more work per thread, overheads like memory access matter less. Single-thread efficiency for the large problem slightly exceeds 1.0, likely due to better cache use.
For $N = 10^6$, speedup reaches about 4× and stays stable up to 16 threads, while $N = 10^5$ levels off around 2.3×. This plateau occurs because the system has 4 physical cores, and memory bandwidth also limits further speedup.

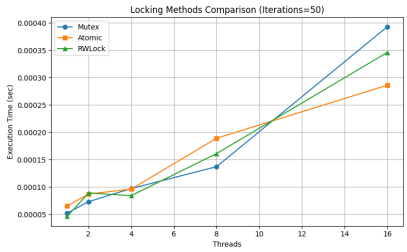
Exercise 1.2

The objective of this exercise is to parallelize the increment of a counter across multiple threads, ensuring that the execution is deterministic and correct, and to compare three different approaches for handling the critical section:

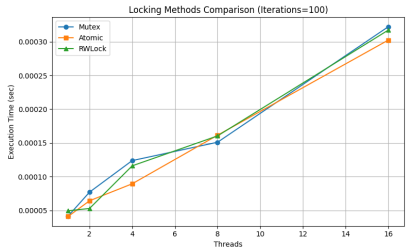
- Using mutual exclusion locks (mutexes)
- Using read-write locks
- Using atomic operations



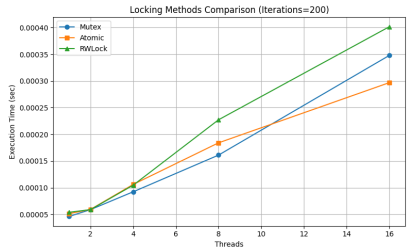
(a) Iterations = 10



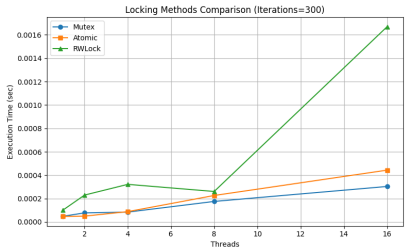
(b) Iterations = 50



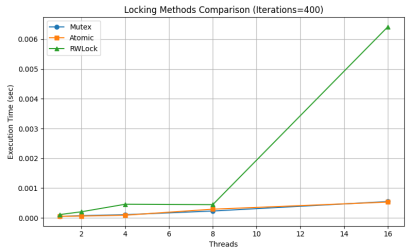
(c) Iterations = 100



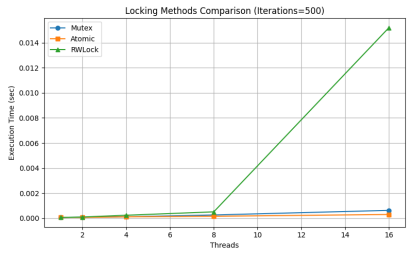
(d) Iterations = 200



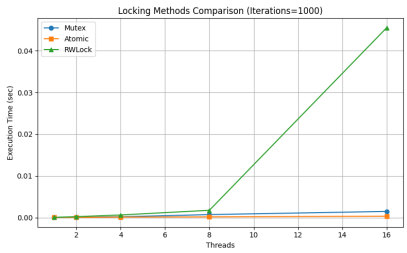
(e) Iterations = 300



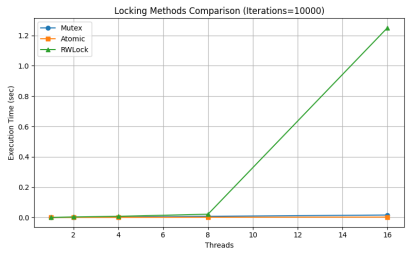
(f) Iterations = 400



(g) Iterations = 500



(h) Iterations = 1000



(i) Iterations = 10000

Figure 3: Comparative plots of the locking methods (Mutex, Atomic, RWLock) for different numbers of threads and iterations.

Analysis of Results

- Small iterations (10–100): Execution times are similar for Mutex, Atomic, and RWLock, with minimal locking overhead.
- Moderate iterations (200–500): Atomic operations are fastest due to low overhead; Mutexes perform reasonably well; RWLocks start to slow down under thread contention.
- Large iterations (1000–10000): Atomic scales very well even with many threads, Mutexes moderately, while RWLocks suffer significantly in write-heavy workloads.

Execution times increase with threads. Atomic shows near-linear scaling, Mutex moderate, RWLocks poor in write-dominated scenarios. So, atomic is the most efficient for shared counters. Mutex is safe and fairly efficient. RWLocks are suitable only for read-heavy workloads.

Exercise 1.3

The objective of this exercise is to parallelize the process of counting non-zero elements in different arrays and storing the results in a given struct. The original struct is prone to false sharing, which can slow down the execution and reduce the observable speedup. On the left is the original struct, and on the right is the modified version with padding to prevent false sharing.

```
struct array_stats_s {
    long long int info_array_0;
    long long int info_array_1;
    long long int info_array_2;
    long long int info_array_3;
} array_stats;
```

(a) Plain structure (no padding)

```
struct array_stats_s {
    long long int info_array_0;
    char pad0[64 - sizeof(long long int)];
    long long int info_array_1;
    char pad1[64 - sizeof(long long int)];
    long long int info_array_2;
    char pad2[64 - sizeof(long long int)];
    long long int info_array_3;
    char pad3[64 - sizeof(long long int)];
} array_stats;
```

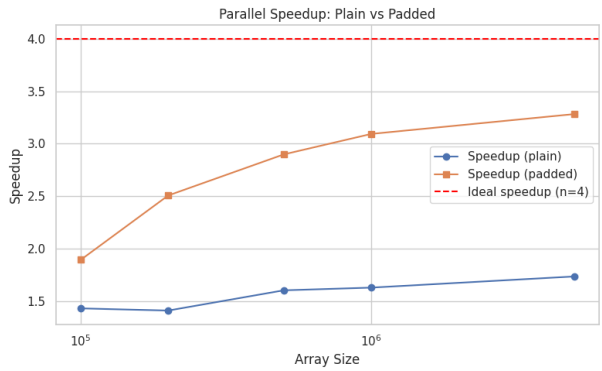
(b) Padded structure (cache-line aligned)

Figure 4: Comparison of plain and padded `array_stats_s` structures.

Analysis of Results

Below are the measurements for the plain and padded versions. It is evident that the implementation with padding achieves near-ideal speedup, approaching the theoretical maximum of 4× as the array size increases.

(a) Sequential vs Parallel Execution Time



(b) Measured vs Ideal Speedup

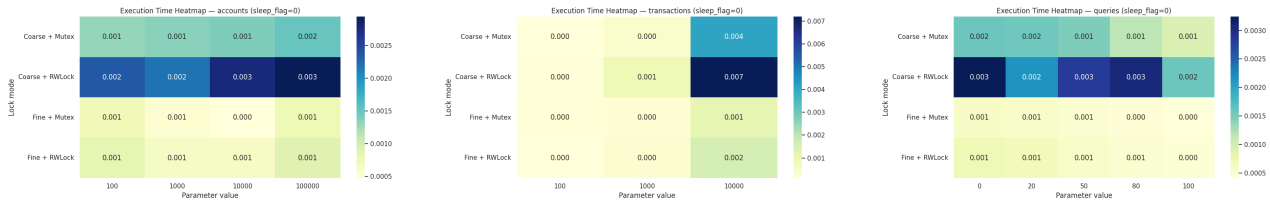
Note: No locking mechanisms were necessary because there were no critical sections. Each thread was assigned a unique counter, which only that thread modifies, avoiding any race conditions.

Exercise 1.4

The objective of this exercise is to implement a bank simulator that supports two types of operations: queries and exchanges. The simulator can use either mutexes or read-write locks, and either coarse- or fine-grained locking. However, since each transaction is very lightweight, the performance differences between locking strategies are negligible when measuring execution time for small workloads.

Analysis of Results

As expected, increasing the number of accounts or transactions per thread leads to a corresponding increase in execution time. Similarly, varying the query-to-transaction ratio results in minimal differences when the critical sections are very short.



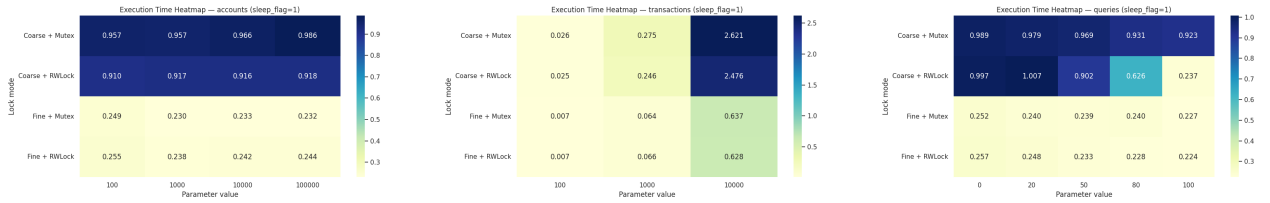
(a) Accounts sweep (b) Transactions sweep (c) Query ratio sweep

Figure 6: Execution time heatmaps with sleep flag = 0 (no sleep in critical sections).

Note: The colors in the heatmaps above might appear to differ significantly, but when comparing the actual color scales, the differences are minimal, close to zero.

For larger critical sections (using sleep), fine-grained locking consistently outperforms coarse-grained locking as the number of accounts increases. This trend is also evident when varying the number of transactions, where increasing the workload significantly affects execution time.

Coarse-grained locking also shows clear advantages when adjusting the query-to-transaction ratio, with one notable exception: when transactions are 100% queries, execution time is similar to fine-grained locking. This occurs because read-heavy workloads allow read-write locks to perform efficiently under coarse locking as well.



(a) Accounts sweep (b) Transactions sweep (c) Query ratio sweep

Figure 7: Execution time heatmaps with sleep flag = 1 (critical sections include microsecond-level sleeps).

Overall, for small critical sections, both coarse- and fine-grained locking perform adequately. For longer critical sections, fine-grained locking generally offers better performance, except in read-dominant scenarios where coarse-grained locking can also be effective.

Exercise 1.5

The objective of this exercise is to experiment with barrier synchronization using three different implementations:

- The standard `pthread_barrier_t` provided by `pthread`.
- A custom implementation based on condition variables, inspired by the `pth_cond_var.c` example from Chapter 4.
- A busy-waiting implementation following the sense-reversal barrier concept [1].

Each thread executes the following loop:

```
for i = 0 to n:
    barrier_wait
```

Thus, all threads are synchronized at each iteration of the barrier.

Analysis of Results

The 3D plots below illustrate the effect of the **number of threads** and the **number of iterations per thread** on execution time.

It is evident from the plots that the number of threads significantly affects execution time. The color map shows that the `pthread` and condition-variable implementations exhibit similar performance. In contrast, the busy-waiting implementation incurs substantially higher execution times, especially when the number of threads exceeds the number of physical cores. This is due to resource contention, which dramatically increases the total execution time.

For small number of threads the execution time is better than the other implementations.

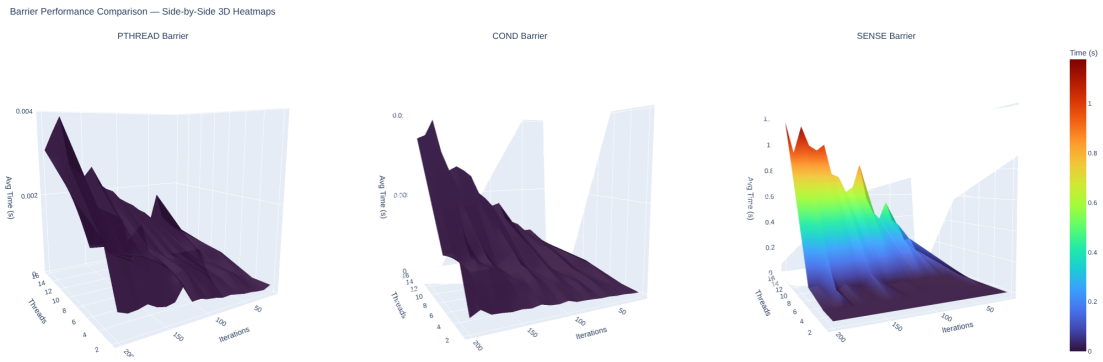


Figure 8: 3D visualization of barrier performance. The X-axis represents the number of threads, the Y-axis represents the number of iterations, and the Z-axis represents the average execution time. The color intensity reflects execution time according to a common scale across all implementations.

The sense-reversal barrier works because each thread waits for a global flag to match its own local sense, flipping it when all threads arrive. The last thread resets the counter and flips the flag, which releases all waiting threads. This mechanism ensures the barrier can be used repeatedly in loops, as each iteration starts fresh.

References

[1] Barrier (computer science). *Wikipedia*. `https://en.wikipedia.org/wiki/Barrier_(computer_science)`.